Multi-Agent Control of a Manufacturing Cell with LLM-Based Order Intake

Gabriel Danho MUA600 Multi-Agent Systems Examiner: Mattias Bennulf June 2026

1 Introduction

Mass customisation and low-volume production force manufacturing systems to be reconfigured far more often than rigidly automated lines allow [1], [2]. Plug & Produce standardises hardware modules so that resources can be added or moved quickly [2], but the control software still usually has to be reprogrammed for each new setup. A Large Language Model (LLM) can reduce this by letting an operator instruct the system in natural language, in line with the human-centric goals of Industry 5.0 [1]. This project implements a multi-agent control system for a simulated manufacturing cell - a transport crane, two sources, two processing stations and a sink, controlled over Modbus TCP - and extends it with such a natural-language order intake. The work covers five requirements (R1-R5): R1-R4 concern the agent system and R5 adds the LLM order intake.

2 System design and implementation

Approach. A hybrid design is used: the agent system is built in CMAS and the LLM planner runs as a separate Python process. CMAS fits the agent system because it provides native Modbus TCP binding on every variable, a built-in negotiation and booking mechanism through abstract interfaces, and an explicit Plug & Produce model with runtime discovery. The LLM runs in Python with a local model (llama3 via Ollama); it produces validated orders that the agent system consumes and never writes to the control layer directly.

Agents. Responsibilities are strictly separated. The Crane owns only the motion signals and a transport skill, with no knowledge of part types or stations. Process1 and Process2 own their status signals, location and a processing skill with a capability descriptor. Source1 and Source2 own a sensor and location and deploy a new part agent when a part appears, and the Sink owns only its location. The product agent, Part, owns its process plan and is the only agent that knows its full route through the cell, requesting transport and processing through abstract interfaces and re-planning on failure. One coordination agent, OrderIntake, is the only order-aware agent and deploys the part agents requested by the LLM.

Communication. All communication uses abstract interfaces with negotiation rather than hard-coded references: a part declares an interface and calls a booking function, CMAS broadcasts the request, compatible stations reply, and one is bound. A booked interface cannot be re-booked, so parts that compete for a station are serialised automatically, giving mutual exclusion with no deadlock or double-booking. Each resource exposes its own coordinate and the crane reads the location of the station it is bound to, so moving a station changes only that station's own data.

Requirements. *R1 (decentralised control):* the crane only transports, the part owns its plan, and each station owns its operation logic, with no shared knowledge. R2 (Plug & Produce): a new product is a new part type with its own plan, and the crane is unchanged because it never sees the part type. R3 (runtime locations): each resource owns its coordinate and the crane queries the bound resource. R4 (failure recovery): a failed station is released and re-booking selects the alternative automatically. R5 (order intake): a validated JSON order drives part deployment and the LLM never writes to the control layer.

LLM order intake (R5). A Python script turns an instruction such as "make three type-1 parts and two type-2 parts" into a JSON order of the form {"orders":[{"part_type":1,"quantity":3}, ...]}, prompting the model with a strict schema and validating the keys, data types and value ranges before accepting it; malformed output is rejected and re-prompted up to a small retry limit. Because CMAS Structured Text has no file or network access, the validated quantities are written to dedicated Modbus holding registers (with a JSON audit log), and the OrderIntake agent deploys the corresponding parts, decrementing each count as a part is released. The validation step is the boundary that keeps the model out of the control path.

3 Results and conclusions

The complete system was tested end to end. The instruction above produced a validated order, and the cell built exactly that mix - three type-1 and two type-2 parts, each routed through its station to the sink, with the counts enforced and the agent logic unchanged; the R1-R4 behaviour (decentralised routing, a second product type with no crane change, runtime locations, and re-routing on a failed station) was confirmed. The validation boundary failed safe: malformed model output was rejected and nothing reached the control layer. The main benefit of the LLM over a hand-written parser is flexibility at the human-facing end, while the strict schema keeps the system deterministic and safe; the main limitations are the human-triggered part release in the simulation, the small local model, and the single-cell scope. In summary, the project meets requirements R1-R5 and demonstrates a decentralised, reconfigurable manufacturing cell that can be instructed in natural language, with the language model kept out of the safety-critical control path.

References

[1] M. Bennulf, F. Danielsson and X. Zhang, "Human-Robot Communication using Large Language Models for Automated Kitting," IOP Conf. Ser.: Mater. Sci. Eng., vol. 1342, art. 012049, 2026.

[2] T. Arai, Y. Aiyama, Y. Maeda, M. Sugi and J. Ota, "Agile Assembly System by Plug and Produce," CIRP Annals, vol. 49, no. 1, pp. 1-4, 2000.

[3] M. Bennulf and F. Danielsson, "Using Large-Language Models for Plug & Produce Manufacturing," 23rd Int. Industrial Simulation Conf. (ISC 2025), EUROSIS, pp. 43-47, 2025.